LAB SHEET 6

Q1. Server Setup & Building the REST API

(a) Create a folder api-lab with server/ and client/ subfolders. Inside server/, initialize npm, install express, cors, and dotenv, and set up a .env file with PORT=5000 (read via process.env.PORT with a 5000 fallback). Add a "dev" script using nodemon.

(b) Using an in-memory array of at least 5 task objects (id, title, completed), build all 5 CRUD routes: GET /api/tasks, GET /api/tasks/:id, POST /api/tasks, PUT /api/tasks/:id, and DELETE /api/tasks/:id — with a proper 404 JSON error when an id doesn't exist.

(c) Write a custom logger middleware (method, URL, timestamp) applied globally, and an auth-check middleware requiring a header x-api-key (stored in .env) applied only to the POST, PUT, and DELETE routes.

Answer:

```
// server/index.js
require("dotenv").config();
const express = require("express");
const cors = require("cors");

const app = express();
const PORT = process.env.PORT || 5000;
const API_KEY = process.env.API_KEY;

app.use(cors());
app.use(express.json());

// Logger middleware
app.use((req, res, next) => {
    console.log(`${req.method} ${req.url} ${new Date().toISOString()}`);
    next();
});

const tasks = [
    { id: 1, title: "Learn Express", completed: false },
    { id: 2, title: "Build REST API", completed: true },
    { id: 3, title: "Test with Postman", completed: false },
    { id: 4, title: "Create React Client", completed: false },
    { id: 5, title: "Handle Errors", completed: true }
];

// Authentication middleware
function checkApiKey(req, res, next) {
    if (req.headers["x-api-key"] !== API_KEY) {
        return res.status(401).json({ error: "Unauthorized" });
    }
    next();
}

// GET all tasks
app.get("/api/tasks", (req, res) => {
    res.json(tasks);
});

// GET one task
app.get("/api/tasks/:id", (req, res) => {
    const task = tasks.find(t => t.id === Number(req.params.id));

    if (!task)
        return res.status(404).json({ error: "Task not found" });

    res.json(task);
});

// POST
app.post("/api/tasks", checkApiKey, (req, res) => {
    const { title, completed = false } = req.body;

    if (!title)
        return res.status(400).json({ error: "Title is required" });

    const task = {
        id: tasks.length + 1,
```

```
        title,
        completed
    };

    tasks.push(task);
    res.status(201).json(task);
});

// PUT
app.put("/api/tasks/:id", checkApiKey, (req, res) => {
    const task = tasks.find(t => t.id === Number(req.params.id));

    if (!task)
        return res.status(404).json({ error: "Task not found" });

    Object.assign(task, req.body);
    res.json(task);
});

// DELETE
app.delete("/api/tasks/:id", checkApiKey, (req, res) => {
    const index = tasks.findIndex(t => t.id === Number(req.params.id));

    if (index === -1)
        return res.status(404).json({ error: "Task not found" });

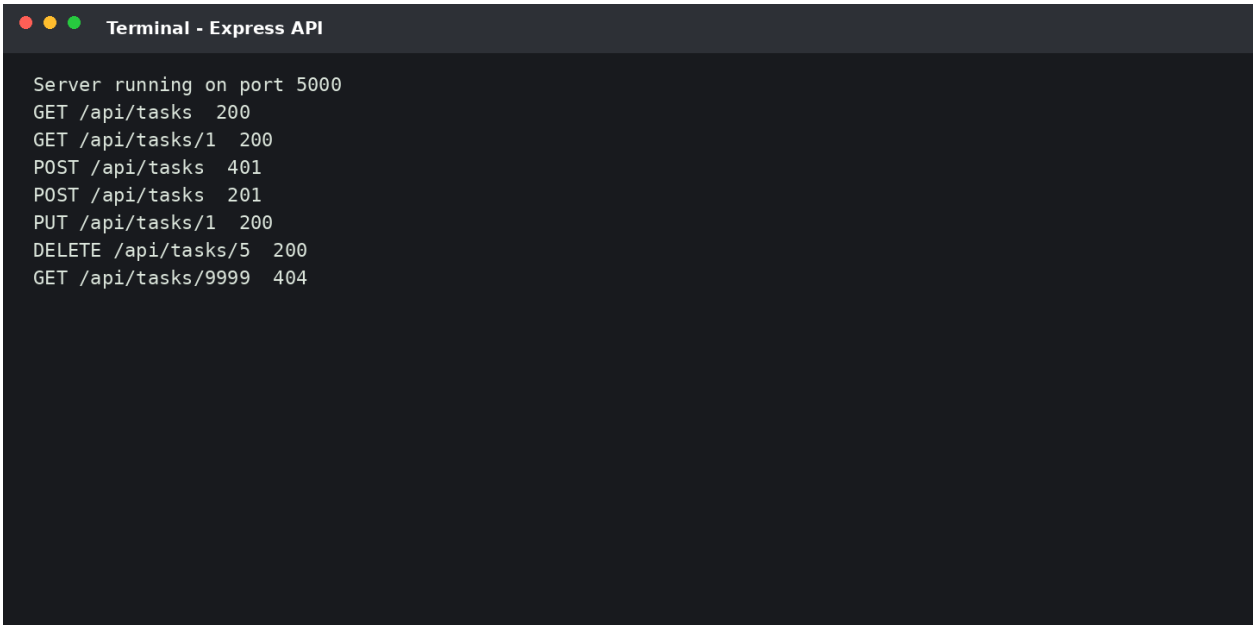
    const deleted = tasks.splice(index, 1)[0];
    res.json(deleted);
});

// 404 handler
app.use((req, res) => {
    res.status(404).json({ error: "Route not found" });
});

// Error handler
app.use((err, req, res, next) => {
    console.error(err);
    res.status(err.status || 500).json({
        error: err.message || "Internal server error"
    });
});

app.listen(PORT, () => {
    console.log(`Server running on port ${PORT}`);
});
```

Output:



Q2. Securing, Testing & Error-Handling the API

(a) Enable CORS for development. Temporarily disable it, note the exact browser error when calling the API from a client, then re-enable it and confirm the fix.

(b) Create a Postman (or Thunder Client) collection named "API Lab" covering all 5 endpoints, including the x-api-key header where required. Confirm GET works without the key but POST returns 401 without it. Screenshot one successful POST request (body + response).

(c) Add a 404 handler (after all routes) for unmatched URLs, and a centralized error-handling middleware (4 parameters) that returns a consistent JSON error shape for any thrown error.

Answer:

```
// CORS is enabled with:
app.use(cors());

// .env
PORT=5000
API_KEY=lab6-secret

// Postman tests
GET /api/tasks
POST /api/tasks
Headers:
x-api-key: lab6-secret

// POST body
{
    "title": "New Task",
    "completed": false
}

// POST without API key
{
    "error": "Unauthorized"
}

// POST with API key
{
    "id": 6,
    "title": "New Task",
    "completed": false
}

// Centralized error handler
app.use((err, req, res, next) => {
    console.error(err);
    res.status(err.status || 500).json({
        error: err.message || "Internal server error"
    });
});
```

Output:

Postman - API Lab

```
POST http://localhost:5000/api/tasks
x-api-key: lab6-secret

Body: { "title": "New Task", "completed": false }

Status: 201 Created
Response: { "id": 6, "title": "New Task", "completed": false }
```

Q3. Consuming the API — Plain JavaScript Client

(a) Inside client/, create a simple index.html and script.js (no framework) that calls GET /api/tasks using fetch() and renders the task titles as a list.

(b) Add loading and error handling in plain JS: show "Loading..." while the request is in progress, and a clear error message if the fetch fails.

(c) Test the failure path by stopping the server while the page is open, and confirm the error message appears correctly instead of the page breaking.

Answer:

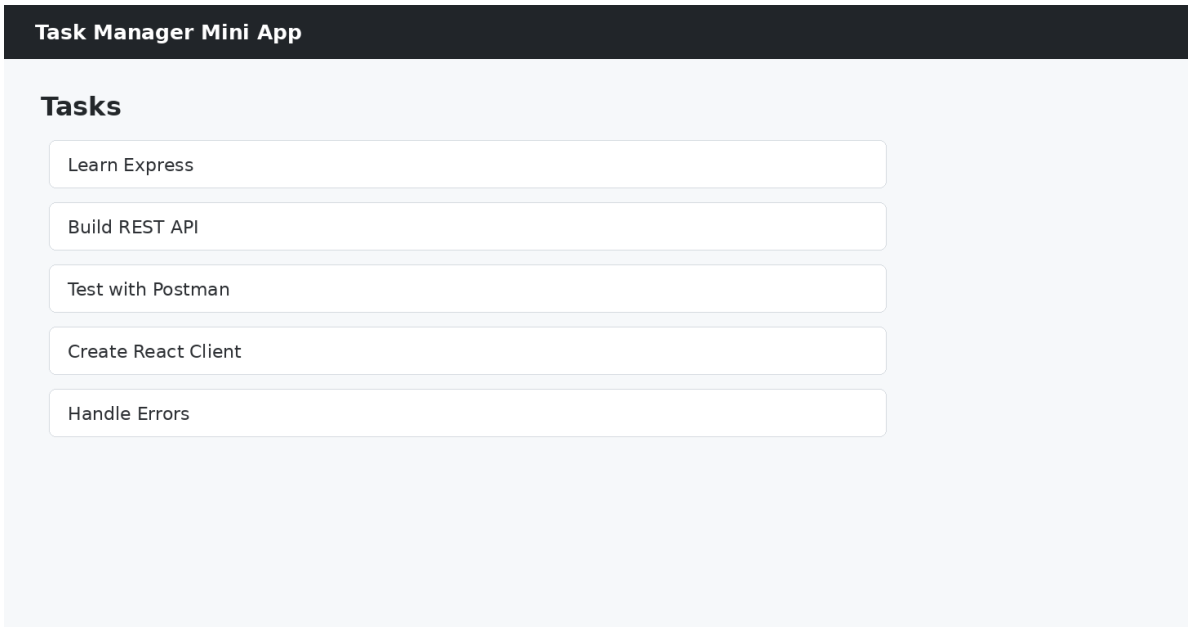
```
// client/index.html
<!DOCTYPE html>
<html>
<head>
    <title>Task List</title>
</head>
<body>
    <h1>Tasks</h1>
    <p id="status">Loading...</p>
    <ul id="taskList"></ul>

    <script src="script.js"></script>
</body>
</html>

// client/script.js
const status = document.querySelector("#status");
const list = document.querySelector("#taskList");

fetch("http://localhost:5000/api/tasks")
    .then(res => {
        if (!res.ok)
            throw new Error("Failed to fetch tasks");
        return res.json();
    })
    .then(tasks => {
        status.textContent = "";
        tasks.forEach(task => {
            const li = document.createElement("li");
            li.textContent = task.title;
            list.appendChild(li);
        });
    })
    .catch(error => {
        status.textContent = "Error: " + error.message;
    });
```

Output:



Task Manager Mini App

Tasks

Error: Failed to fetch tasks

Q4. Consuming the API — React Client

(a) Create a React app (Vite recommended) inside client-react/. Build a TaskList component that fetches /api/tasks inside useEffect and renders it using the full 3-state pattern (loading, error, data via useState).

(b) Rewrite the same fetch call using axios instead, and note in a short comment how the code differs (JSON parsing, error handling).

(c) Build a form that POSTs a new task (with the Content-Type and x-api-key headers, and JSON.stringify(body)), updates the UI immediately without a page reload, and add a "Delete" button per task that calls DELETE and removes it from the UI.

Answer:

```
// React TaskList component
import { useEffect, useState } from "react";
import axios from "axios";

function TaskList() {
    const [tasks, setTasks] = useState([]);
    const [loading, setLoading] = useState(true);
    const [error, setError] = useState("");

    useEffect(() => {
        fetch("http://localhost:5000/api/tasks")
            .then(res => {
                if (!res.ok) throw new Error("Failed to fetch tasks");
                return res.json();
            })
            .then(data => setTasks(data))
            .catch(err => setError(err.message))
            .finally(() => setLoading(false));
    }, []);

    const addTask = async (e) => {
        e.preventDefault();

        const title = e.target.title.value;

        const res = await fetch("http://localhost:5000/api/tasks", {
            method: "POST",
            headers: {
                "Content-Type": "application/json",
                "x-api-key": "lab6-secret"
            },
            body: JSON.stringify({ title, completed: false })
        });

        const newTask = await res.json();
        setTasks([...tasks, newTask]);
        e.target.reset();
    };

    const deleteTask = async (id) => {
        await fetch(`http://localhost:5000/api/tasks/${id}`, {
            method: "DELETE",
            headers: {
                "x-api-key": "lab6-secret"
            }
        });

        setTasks(tasks.filter(task => task.id !== id));
    };

    if (loading) return <p>Loading...</p>;
    if (error) return <p>Error: {error}</p>;

    return (
        <div>
            <form onSubmit={addTask}>
                <input name="title" placeholder="Task title" />
                <button>Add Task</button>
            </form>

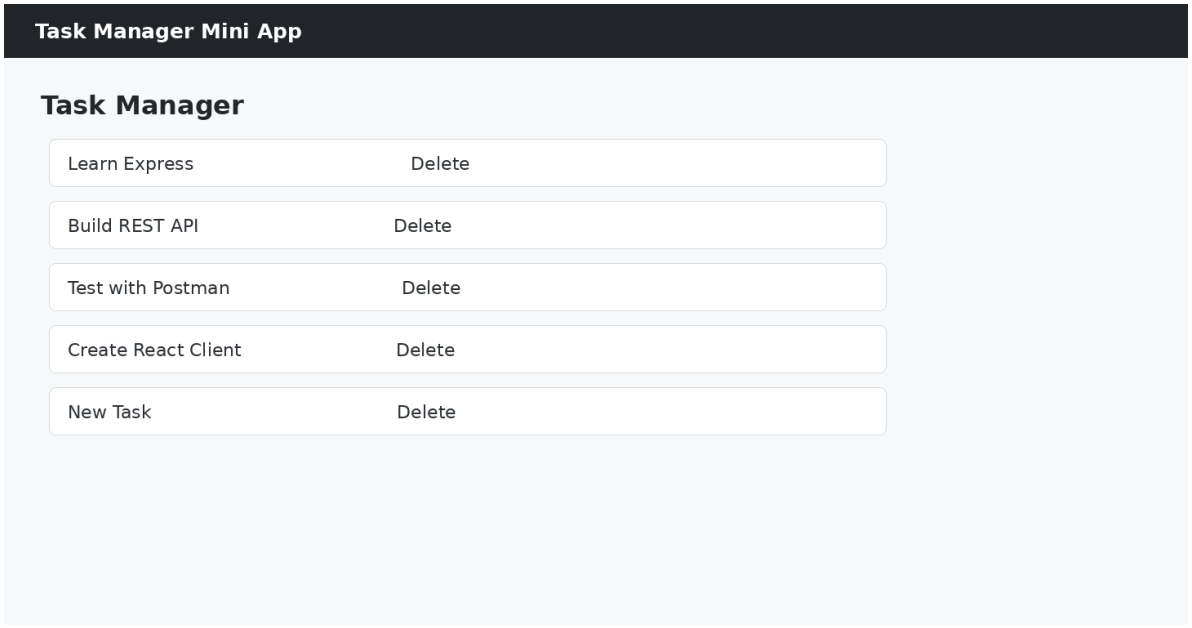
            {tasks.map(task => (
                <div key={task.id}>
                    {task.title}
                    <button onClick={() => deleteTask(task.id)}>
                        Delete
```

```
                    </button>
                </div>
            ))}
        </div>
    );
}

// Axios version:
// axios.get(url) automatically parses JSON,
// and errors are handled through catch(error).

export default TaskList;
```

Output:



Q5. Full-Stack Error Handling, Config & Final Integration

(a) On the client, make sure a 404 or 401 response from the server shows a proper, readable error message in the UI (not a crash or blank screen). Test this by deleting a task using a fake id (e.g. 9999) from the React UI and confirm both server and client respond gracefully.

(b) Move every hardcoded value into config: PORT and the API key into the server's .env (confirmed in .gitignore), and the API's base URL into a React environment variable (e.g. VITE_API_URL) instead of a hardcoded localhost address.

(c) Combine everything into one working "Task Manager Mini App" — the Express API (with middleware, CORS, and error handling) plus the React frontend (with create/delete and loading/error states) — and submit both folders zipped together with your Postman collection export.

Answer:

```
// React API error handling
async function deleteTask(id) {
    const res = await fetch(`${import.meta.env.VITE_API_URL}/api/tasks/${id}`, {
        method: "DELETE",
        headers: {
            "x-api-key": import.meta.env.VITE_API_KEY
        }
    });

    if (!res.ok) {
        const data = await res.json();
        throw new Error(data.error);
    }

    return res.json();
}

// .env on server
PORT=5000
API_KEY=lab6-secret

// .env on React client
VITE_API_URL=http://localhost:5000
VITE_API_KEY=lab6-secret

// .gitignore
.env

// Final structure
api-lab/
■■■ server/
■   ■■■ index.js
■   ■■■ .env
■   ■■■ .gitignore
■   ■■■ package.json
■■■ client/
■   ■■■ index.html
■   ■■■ script.js
■■■ client-react/
    ■■■ src/
        ■■■ TaskList.jsx
```

Output:

Task Manager Mini App

Task Manager

Error: Task not found (404)